

Types for Programs and Proofs

Operational semantics, type safety, and a correct compiler

Thierry Coquand

Plan

First hour — arith1.agda: a very small programming language, and what types are *for*

- the syntax, and the *one step* relation $e \Rightarrow e'$
- values, and expressions which get *stuck*
- the typing relation $e :: T$; *preservation* and *progress*

Second hour — operational.agda: *big step* semantics $e \Downarrow v$, and how it relates to $\Rightarrow$

compiler.agda: a *compiler* to a stack machine, and its correctness proof

Where we are

Lecture 5: *indexed inductive families* — a family $R : A \rightarrow \text{Set}$ whose constructors are the rules of a relation

Lecture 6: a *type theory* is a raw syntax, a list of judgment forms, and a *finite list of rules*; each rule is a constructor

Today we use exactly the same device on something much smaller than type theory: a language with booleans and natural numbers

three relations: $e \Rightarrow e'$ $e \Downarrow v$ $e :: T$

and we *prove theorems* about a programming language, in Agda

The language

```
data Exp : Set where
  true   : Exp
  false  : Exp
  zero   : Exp
  suc    : Exp -> Exp
  if     : Exp -> Exp -> Exp -> Exp
  pred   : Exp -> Exp
  isZ    : Exp -> Exp

exp0 : Exp
exp0 = if (isZ zero) zero (pred zero)
```

This is the *raw syntax*: `suc true` and `isZ false` are expressions. Nothing here says what an expression *means*, or which expressions make sense

What is an operational semantics?

We say what a program *does*, not what it denotes: we define a relation

$$e \Rightarrow e' \quad \text{“}e \text{ computes to } e' \text{ in one step”}$$

by a finite list of rules. This is *structural* operational semantics: the rules follow the structure of the expression

G. Plotkin, *A structural approach to operational semantics*, Aarhus 1981; and *The Origins of Structural Operational Semantics*, 2004

In Agda: an inductive family $_ \Rightarrow _ : \text{Exp} \rightarrow \text{Exp} \rightarrow \text{Set}$, one constructor per rule

The rules which do the work

```
data _=>_ : Exp -> Exp -> Set where
```

<pre>=>true : {e0 e1 : Exp} -> ----- if true e0 e1 => e0</pre>	<pre>=>false : {e0 e1 : Exp} -> ----- if false e0 e1 => e1</pre>
---	---

<pre>=>isZ0 : ----- isZ zero => true</pre>	<pre>=>isZS : {e : Exp} -> ----- isZ (suc e) => false</pre>
--	--

<pre>=>pred0 : ----- pred zero => zero</pre>	<pre>=>predS : {e : Exp} -> ----- pred (suc e) => e</pre>
--	--

Six *computation* rules. Each one has no premise: it is an axiom

The rules which say *where* to work

$\Rightarrow \text{if} : \{e \ e' \ e_0 \ e_1 : \text{Exp}\} \rightarrow e \Rightarrow e' \rightarrow$

$\text{if } e \ e_0 \ e_1 \Rightarrow \text{if } e' \ e_0 \ e_1$

$\Rightarrow \text{suc} : \{e \ e' : \text{Exp}\} \rightarrow e \Rightarrow e' \rightarrow$

$\text{suc } e \Rightarrow \text{suc } e'$

$\Rightarrow \text{isZ} : \{e \ e' : \text{Exp}\} \rightarrow e \Rightarrow e' \rightarrow$

$\text{isZ } e \Rightarrow \text{isZ } e'$

$\Rightarrow \text{pred} : \{e \ e' : \text{Exp}\} \rightarrow e \Rightarrow e' \rightarrow$

$\text{pred } e \Rightarrow \text{pred } e'$

Four *congruence* rules. They fix the *strategy*: in $\text{if } e \ e_0 \ e_1$ one may only reduce e , never e_0 or e_1

There is *no* rule for $\text{if } e \ e_0 \ e_1 \Rightarrow \text{if } e \ e_0' \ e_1$. A different choice of congruence rules is a different language

A step is a term

```
test=> : exp0 => if true zero (pred zero)
test=> = => if => isZ0
```

Reading it from the inside out:

$$\frac{\frac{}{\text{isZ zero} \Rightarrow \text{true}} \Rightarrow \text{isZ0}}{\text{if (isZ zero) zero (pred zero)} \Rightarrow \text{if true zero (pred zero)}} \Rightarrow \text{if}$$

The derivation *tree* and the Agda *term* are the same object. Agda checks the derivation for us

Many steps

```
data _=>*_ : Exp -> Exp -> Set where
  nil  : {e : Exp} -> e =>* e
  cons : {e0 e1 e2 : Exp} -> e0 => e1 -> e1 =>* e2 -> e0 =>* e2

isTrans : {e0 e1 e2 : Exp} -> e0 =>* e1 -> e1 =>* e2 -> e0 =>* e2
isTrans nil      q = q
isTrans (cons x p) q = cons x (isTrans p q)

test=>* : exp0 =>* zero
test=>* = cons (=>if =>isZ0) (cons =>true nil)
```

An element of $e0 \Rightarrow^* e1$ *is a list* of steps: it is the type of lists, indexed so that the ends match. Transitivity *is* concatenation

The values

```
data isValnat : Exp -> Set where
  isValnat0 : isValnat zero
  isValnatS : {e : Exp} -> isValnat e -> isValnat (suc e)
```

```
data isVal : Exp -> Set where
  Valtrue  : isVal true
  Valfalse : isVal false
  Valnat   : {e : Exp} -> isValnat e -> isVal e
```

A value is an expression we accept as an *answer*: the two booleans and the numerals zero, suc zero, . . .

Note that suc true is *not* a value, though it is an expression

A value never moves

```
=>normal : Exp -> Set
=>normal e = {e1 : Exp} ->  $\neg$  (e => e1)

lem0 : {e : Exp} -> isValnat e -> =>normal e
lem0 isValnat0      ()
lem0 (isValnatS ie) (=>suc h) = lem0 ie h

lem1 : {e : Exp} -> isVal e -> =>normal e
lem1 Valtrue      ()
lem1 Valfalse     ()
lem1 (Valnat x) = lem0 x
```

`()` is the *absurd pattern*: there is no constructor of `_=>_` whose conclusion is `zero => e1`, so there is nothing to write

Is the converse true?

Is every normal form a value? Is every expression either a value or reducible?

We can *state* this. We need a disjunction and an existential:

```
data _+_ (A B : Type) : Type where
  inl : A -> A + B
  inr : B -> A + B

record  $\Sigma$  (A : Type) (B : A -> Type) : Type where
  constructor _,_
  field
    fst : A
    snd : B fst

progress : Exp -> Set
progress e = isVal e +  $\Sigma$  Exp ( $\lambda$  e1 -> e ==> e1)
```

Some expressions are stuck

```
isStuck : Exp -> Set
isStuck e =  $\neg$  (progress e)

remark : isStuck (isZ true)
remark (inl (Valnat ()))
remark (inr (_ , =>isZ ()))
```

`isZ true` is not a value, and no rule applies to it: the machine *halts on an error*. Two lines, both absurd patterns

In a real machine this is a segmentation fault, or worse, a wrong answer. We would like to *rule these expressions out before running them*

Types

Types appeared first in *logic*, not in programming: to prevent the paradoxes.
B. Russell, 1901 → 1908

The alternative was to put down *axioms* and hope they are consistent

In programming the idea is the same: a *cheap, decidable* test, done before running the program, which rejects some expressions — among them *all* the ones that would get stuck

The price is that some harmless expressions are rejected too

The typing relation

```
data Typ : Set where
  nat  : Typ
  bool : Typ

data _::_ : Exp -> Typ -> Set where
  typet  : true :: bool
  typef  : false :: bool
  type0  : zero :: nat
  typeS  : {e : Exp} -> e :: nat -> suc e :: nat
  typep  : {e : Exp} -> e :: nat -> pred e :: nat
  typeZ  : {e : Exp} -> e :: nat -> isZ e :: bool
  typeif : {T : Typ} -> {e e0 e1 : Exp} ->
    e :: bool -> e0 :: T -> e1 :: T -> if e e0 e1 :: T
```

We cannot write $_ :: _$ in Agda, so the judgment $e : T$ is $e :: T$. Yet another inductive family: *seven rules*

A typing derivation is a term

```
exp0N : exp0 :: nat
exp0N = typeif (typeZ type0) type0 (typep type0)
```

$$\frac{\frac{\text{zero} : \text{nat}}{\text{isZ zero} : \text{bool}} \quad \text{zero} : \text{nat} \quad \frac{\text{zero} : \text{nat}}{\text{pred zero} : \text{nat}}}{\text{if (isZ zero) zero (pred zero)} : \text{nat}}$$

Note that `typeif` forces the two branches to have *the same* type: this is what makes `if` safe

Uniqueness of typing

```
uniqueness : (e : Exp) -> {T T' : Typ} -> e :: T -> e :: T' -> T ≡ T'
uniqueness true      typet      typet      = refl
uniqueness false     typef      typef      = refl
uniqueness zero      type0      type0      = refl
uniqueness (suc _)   (typeS _) (typeS _) = refl
uniqueness (pred _)  (typep _) (typep _) = refl
uniqueness (isZ _)   (typeZ _) (typeZ _) = refl
uniqueness (if _ e0 _) (typeif _ T0 _) (typeif _ T0' _) =
    uniqueness e0 T0 T0'
```

One clause per shape of e . Agda sees that only *one* rule can conclude $\text{suc } e :: T$, so the other combinations need not be written

What we want to prove

Preservation (subject reduction): if $e :: T$ and $e \Rightarrow e'$ then $e' :: T$

Progress: if $e :: T$ then e is a value or $e \Rightarrow e_1$ for some e_1

Together: a well typed expression never gets stuck, however long we run it

These are the two halves of what is now called *type safety*. Neither one alone suffices

Preservation

```
preservation : {e e' : Exp} -> {T : Typ} ->
  e :: T -> e => e' -> e' :: T

preservation (typeif typet eT0 eT1) =>true  = eT0
preservation (typeif typef eT0 eT1) =>false = eT1
preservation (typeZ type0)           =>isZ0  = typet
preservation (typeZ (typeS eT))      =>isZS  = typef
preservation (typep type0)           =>pred0  = type0
preservation (typep (typeS eT))      =>predS  = eT
```

The six computation rules. Each is a *one line* argument: for `=>predS` the typing of `pred (suc e)` already contains the typing of `e`, and we simply hand it back

Preservation, the congruences

```
preservation (typeif eT eT0 eT1) (=>if ee') =  
    typeif (preservation eT ee') eT0 eT1  
preservation (typeS eT) (=>suc ee') = typeS (preservation eT ee')  
preservation (typeZ eT) (=>isZ ee') = typeZ (preservation eT ee')  
preservation (typep eT) (=>pred ee') = typep (preservation eT ee')
```

Here the proof is a *recursive call*: this is induction on the derivation of $e \Rightarrow e'$

The shape of the proof follows the shape of the rules exactly. Ten rules, ten clauses

The converse is false

```
exp1 : Exp
exp1 = if true zero false

remark7 : exp1 => zero
remark7 = =>true

remark8 :  $\neg$  (exp1 :: nat)
remark8 (typeif typet type0 ())
```

if true zero false reduces to zero, which has type nat; but the expression itself has *no* type

Subject expansion fails: the types are lost, not gained, by reduction. A type checker is necessarily incomplete

Canonical forms

```
data isBoolVal : Exp -> Set where
  isTrue  : isBoolVal true
  isFalse : isBoolVal false

canon-bool : {e : Exp} -> e :: bool -> isVal e -> isBoolVal e
canon-bool _ Valtrue  = isTrue
canon-bool _ Valfalse = isFalse
canon-bool () (Valnat isValnat0)
canon-bool () (Valnat (isValnatS _))

canon-nat : {e : Exp} -> e :: nat -> isVal e -> isValnat e
canon-nat () Valtrue
canon-nat () Valfalse
canon-nat _ (Valnat v) = v
```

A value of type bool is true or false; a value of type nat is a numeral

Progress, one construction at a time

```
if-val : {u e0 e1 : Exp} -> isBoolVal u -> progress (if u e0 e1)
if-val {e0 = e0} isTrue  = inr (e0 , =>true)
if-val {e1 = e1} isFalse = inr (e1 , =>false)
```

```
pred-val : {u : Exp} -> isValnat u -> progress (pred u)
pred-val isValnat0      = inr (zero , =>pred0)
pred-val (isValnatS {e} _) = inr (e , =>predS)
```

```
isZ-val : {u : Exp} -> isValnat u -> progress (isZ u)
isZ-val isValnat0      = inr (true , =>isZ0)
isZ-val (isValnatS _) = inr (false , =>isZS)
```

If the subexpression is already a value, the whole expression moves. Canonical forms is what tells us *which* rule applies

Progress, the two cases

```
typeprog-if : {u e0 e1 : Exp} ->
  u :: bool -> progress u -> progress (if u e0 e1)
typeprog-if uT (inl v) = if-val (canon-bool uT v)
typeprog-if {e0 = e0} {e1} uT (inr (u' , x)) =
  inr (if u' e0 e1 , =>if x)
```

```
typeprog-pred : {u : Exp} ->
  u :: nat -> progress u -> progress (pred u)
typeprog-pred uT (inl v) = pred-val (canon-nat uT v)
typeprog-pred uT (inr (u' , x)) = inr (pred u' , =>pred x)
```

Either the subexpression *moves*, and the congruence rule carries the step up;
or it is a *value*, and a computation rule fires

typeprog-isZ is the same two lines. typeprog-suc is the one exception:
there a value stays a value, suc of a numeral being one

Progress

```
typeprog : {e : Exp} -> {T : Typ} -> e :: T -> progress e
typeprog typeT          = inl Valtrue
typeprog typeF          = inl Valfalse
typeprog type0          = inl (Valnat isValnat0)
typeprog (typeS eT)     = typeprog-suc eT (typeprog eT)
typeprog (typeP eT)     = typeprog-pred eT (typeprog eT)
typeprog (typeZ eT)     = typeprog-isZ eT (typeprog eT)
typeprog (typeIf eT eT0 eT1) = typeprog-if eT (typeprog eT)
```

Structural induction on the *typing* derivation: seven rules, seven clauses, and each recursive call is on a *smaller derivation*

Compare: `typeprog eT` is used twice, once as a proof and once as the argument telling us which case we are in

Safety

```
theorem : {e e' : Exp} -> {T : Typ} ->  
          e :: T -> e =>* e' -> progress e'  
theorem eT nil          = typeprog eT  
theorem eT (cons x q) = theorem (preservation eT x) q
```

Two lines. The induction is on the *length of the computation*: at each step preservation restores the hypothesis, and progress does the rest

Z. Manna called this *computational induction*; Manna, Ness, Vuillemin, *Inductive methods for proving properties of programs*, 1973

“Well typed programs cannot go wrong”

R. Milner, *A theory of type polymorphism in programming*, 1978

Read the slogan carefully. It does *not* say the program terminates — here it happens to, but for a language with recursion it would not

It does *not* say the program is correct

It says exactly: the machine never reaches a state where it does not know what to do. *Safety = progress + preservation*, and nothing more

The converse, at last

```
lem2 : {e : Exp} -> {T : Typ} -> e ofTyp T -> =>normal e -> isVal e
lem2 typet ne = Valtrue
lem2 typef ne = Valfalse
lem2 type0 ne = Valnat isValnat0
lem2 (typeS eT) ne =
  Valnat (isValnatS (canon-nat eT (lem2 eT ( $\lambda$  h -> ne (=>suc h)))))
lem2 (typep eT) ne =
  explosion (pred-red (canon-nat eT (lem2 eT ( $\lambda$  h -> ne (=>pred h))))) ne)
lem2 (typeif eT eT0 eT1) ne =
  explosion (if-red (canon-bool eT (lem2 eT ( $\lambda$  h -> ne (=>if h))))) ne)
```

For a *well typed* expression, normal form and value coincide (typeZ is omitted here; it is like typep)

This is in `arithexp.agda`, where progress is written as an *inductive family* instead of with Σ and $+$

First hour: what to remember

- a semantics is a *relation*, defined by a finite list of rules; in Agda, an indexed inductive family
- computation rules do the work, *congruence rules choose the strategy*
- without types, an expression can be *stuck*: neither a value nor reducible
- *preservation*: induction on the reduction; *progress*: induction on the typing, using *canonical forms*
- safety is their sum, proved by induction on the length of the computation — and it says only that

Exercises, first hour

1. Show that $\Rightarrow$ is *deterministic*: if $e \Rightarrow e_1$ and $e \Rightarrow e_2$ then $e_1 \equiv e_2$
2. Deduce that a normal form reached from e is unique
3. Add a congruence rule $\text{if } e \ e_0 \ e_1 \Rightarrow \text{if } e \ e_0' \ e_1$ for $e_0 \Rightarrow e_0'$. Which of the theorems above still hold?
4. Add $\text{suc} : \text{Exp} \rightarrow \text{Exp}$ at type bool , say $\text{typeS}' : e :: \text{bool} \rightarrow \text{suc } e :: \text{bool}$. Where does the proof of progress break?
5. Show that every well typed expression *terminates*: $e :: T$ implies $e \Rightarrow^* v$ with $\text{isVal } v$. What is the induction?

Second hour

The same language, and then another one

- *big step* semantics: $e \Downarrow v$, “e evaluates to v”. One derivation instead of a sequence of states
- the two semantics agree: $e \Downarrow v$ implies $e \Rightarrow^* v$
- a *compiler* from expressions to a stack machine, and the proof that it is correct
- what makes such a proof go through: finding the statement which the *induction* can carry

Files: `operational.agda` and `compiler.agda`

Big step semantics

Instead of “one step of the machine” we define directly

$e \Downarrow v$ “ e evaluates to the value v ”

No intermediate state is ever mentioned: a derivation of $e \Downarrow v$ is the *whole computation, as one tree*

This is what G. Kahn called *natural semantics* (1987), and it is how an interpreter is naturally written

The rules are *compositional*: to evaluate $\text{pred } e$, evaluate e , then look at the answer

The rules of $\Downarrow$

data $_ \Downarrow _ : \text{Exp} \rightarrow \text{Exp} \rightarrow \text{Set}$ where

$\text{vtrue} : \text{true} \Downarrow \text{true}$

$\text{vfalse} : \text{false} \Downarrow \text{false}$

$\text{v0} : \text{zero} \Downarrow \text{zero}$

$\Downarrow \text{suc} : \{e\ v : \text{Exp}\} \rightarrow e \Downarrow v \rightarrow \text{suc } e \Downarrow \text{suc } v$

$\Downarrow \text{isZ0} : \{e : \text{Exp}\} \rightarrow e \Downarrow \text{zero} \rightarrow \text{isZ } e \Downarrow \text{true}$

$\Downarrow \text{isZS} : \{e\ v : \text{Exp}\} \rightarrow e \Downarrow \text{suc } v \rightarrow \text{isZ } e \Downarrow \text{false}$

$\Downarrow \text{pred0} : \{e : \text{Exp}\} \rightarrow e \Downarrow \text{zero} \rightarrow \text{pred } e \Downarrow \text{zero}$

$\Downarrow \text{predS} : \{e\ v : \text{Exp}\} \rightarrow e \Downarrow \text{suc } v \rightarrow \text{pred } e \Downarrow v$

$\Downarrow \text{iftrue} : \{e\ e0\ e1\ v : \text{Exp}\} \rightarrow e \Downarrow \text{true} \rightarrow e0 \Downarrow v \rightarrow \text{if } e\ e0\ e1 \Downarrow v$

$\Downarrow \text{iffalse} : \{e\ e0\ e1\ v : \text{Exp}\} \rightarrow e \Downarrow \text{false} \rightarrow e1 \Downarrow v \rightarrow \text{if } e\ e0\ e1 \Downarrow v$

Compare the two

Small step, for `exp0`: a *sequence*

```
test=>* : exp0 =>* zero
test=>* = cons (=>if =>isZ0) (cons =>true nil)
```

Big step: a single *tree*

```
test⇓ : exp0 ⇓ zero
test⇓ = ⇓iftrue (⇓isZ0 v0) v0
```

Note $\Downarrow\text{iftrue}$: the branch *not* taken is never evaluated. In the rule this is visible as an absent premise

Big step has the same disease

```
↓valnat : {e : Exp} -> isValnat e -> e ↓↓ e  
↓valnat isValnat0      = v0  
↓valnat (isValnatS v) = ↓↓suc (↓valnat v)
```

```
↓val : {e : Exp} -> isVal e -> e ↓↓ e  
↓val Valtrue      = vtrue  
↓val Valfalse     = vfalse  
↓val (Valnat v) = ↓↓valnat v
```

Values evaluate to themselves. But without types the *result* of ↓↓ need not be a value: `suc true ↓↓ suc true` by `↓↓suc vtrue`

And `isZ true` has no value at all — *the counterpart of being stuck is having no derivation*

What $\Rightarrow^*$ inherits

```
 $\Rightarrow^* \text{suc} : \{e \ e' : \text{Exp}\} \rightarrow e \Rightarrow^* e' \rightarrow \text{suc } e \Rightarrow^* \text{suc } e'$   
 $\Rightarrow^* \text{suc nil} = \text{nil}$   
 $\Rightarrow^* \text{suc (cons x p)} = \text{cons } (\Rightarrow^* \text{suc } x) (\Rightarrow^* \text{suc } p)$ 
```

```
 $\Rightarrow^* \text{if} : \{e \ e' \ e0 \ e1 : \text{Exp}\} \rightarrow e \Rightarrow^* e' \rightarrow \text{if } e \ e0 \ e1 \Rightarrow^* \text{if } e' \ e0 \ e1$   
 $\Rightarrow^* \text{if nil} = \text{nil}$   
 $\Rightarrow^* \text{if (cons x p)} = \text{cons } (\Rightarrow^* \text{if } x) (\Rightarrow^* \text{if } p)$ 
```

Each congruence rule of $\Rightarrow$ lifts to $\Rightarrow^*$, by induction on the list of steps. It is exactly map

Same two lines for $\Rightarrow^* \text{pred}$ and $\Rightarrow^* \text{isZ}$

The two semantics agree

$\text{sound}\Downarrow : \{e \ v : \text{Exp}\} \rightarrow e \Downarrow v \rightarrow e \Rightarrow^* v$

$\text{sound}\Downarrow \text{vtrue} = \text{nil}$

$\text{sound}\Downarrow \text{vfalse} = \text{nil}$

$\text{sound}\Downarrow \text{v0} = \text{nil}$

$\text{sound}\Downarrow (\Downarrow \text{suc } p) = \Rightarrow^* \text{suc } (\text{sound}\Downarrow p)$

$\text{sound}\Downarrow (\Downarrow \text{isZ0 } p) = \text{isTrans } (\Rightarrow^* \text{isZ } (\text{sound}\Downarrow p)) (\text{oneStep} \Rightarrow \text{isZ0})$

$\text{sound}\Downarrow (\Downarrow \text{isZS } p) = \text{isTrans } (\Rightarrow^* \text{isZ } (\text{sound}\Downarrow p)) (\text{oneStep} \Rightarrow \text{isZS})$

$\text{sound}\Downarrow (\Downarrow \text{pred0 } p) = \text{isTrans } (\Rightarrow^* \text{pred } (\text{sound}\Downarrow p)) (\text{oneStep} \Rightarrow \text{pred0})$

$\text{sound}\Downarrow (\Downarrow \text{predS } p) = \text{isTrans } (\Rightarrow^* \text{pred } (\text{sound}\Downarrow p)) (\text{oneStep} \Rightarrow \text{predS})$

$\text{sound}\Downarrow (\Downarrow \text{iftrue } p \ q) = \text{isTrans } (\Rightarrow^* \text{if } (\text{sound}\Downarrow p)) (\text{cons} \Rightarrow \text{true } (\text{sound}\Downarrow q))$

$\text{sound}\Downarrow (\Downarrow \text{iffalse } p \ q) = \text{isTrans } (\Rightarrow^* \text{if } (\text{sound}\Downarrow p)) (\text{cons} \Rightarrow \text{false } (\text{sound}\Downarrow q))$

Induction on the derivation of $e \Downarrow v$: each big step is *replayed* as “reduce inside, then take the computation step”

Which semantics for which purpose?

- *small step* distinguishes non termination from error — an infinite reduction sequence is not a stuck state. It is what one needs for concurrency, and for the safety theorem above
- *big step* is closer to an interpreter, and its proofs are shorter — but a program that loops forever simply has *no* derivation, like a program that fails
- the converse of sound $\Downarrow$ also holds here. It needs: $e \Rightarrow e'$ and $e' \Downarrow v$ imply $e \Downarrow v$

A second question: is the compiler right?

A. Turing, *Checking a large routine*, 1949 — the first paper on proving a program correct

J. McCarthy and J. Painter, *Correctness of a compiler for arithmetic expressions*, 1967 — the example we shall do

R. Milner and R. Weyhrauch, 1972 — the same proof, *machine checked*, in Stanford LCF. One of the first real verifications

X. Leroy, CompCert, 2006 →; CakeML. A verified C compiler, in Coq, and a verified ML compiler, in HOL

The source language

```
data Fin : Nat → Set where
  zero : {n : Nat} → Fin (suc n)
  suc  : {n : Nat} → Fin n → Fin (suc n)
```

```
Vec : Set → Nat → Set
Vec A zero    = ⊤
Vec A (suc n) = A × Vec A n
```

```
data Exp (n : Nat) : Set where
  lit : Nat → Exp n
  var : Fin n → Exp n
  add : Exp n → Exp n → Exp n
```

Exp n is the type of expressions with *at most n free variables*: the index makes “no variable out of scope” a *typing* statement, as in lecture 5

A denotational semantics

```
lookup : {A : Set}{n : Nat} → Fin n → Vec A n → A
lookup zero    (x , xs) = x
lookup (suc i) (x , xs) = lookup i xs
```

```
eval : {n : Nat} → Exp n → Vec Nat n → Nat
eval (lit k)    ρ = k
eval (var i)    ρ = lookup i ρ
eval (add e f)  ρ = eval e ρ + eval f ρ
```

This time the semantics is a *function*, not a relation: it is total, and it is defined by structural recursion

The meaning of an expression is a function from environments to numbers

An example

```
-- the expression (x + 1) + x, with x the only variable
exmp : Exp (suc zero)
exmp = add (add (var zero) (lit one)) (var zero)
```

```
testeval : eval exmp (two , tt)  $\equiv$  suc (suc (suc two))
testeval = refl
```

refl type checks *because Agda computes*: both sides reduce to the same numeral. Nothing to prove

The target: a stack machine

```
data Code (n : Nat) : Set where
  PUSH : Nat    -> Code n → Code n
  LOAD : Fin n -> Code n → Code n
  ADD  :          Code n -> Code n
  HALT :          Code n
```

```
Stack = List Nat
```

```
record State : Set where
  constructor state
  field
    n      : Nat
    instr  : Code n
    env    : Vec Nat n
    stack  : Stack
```

A code is a *list of instructions*; HALT ends it

The machine, as an operational semantics

```
data _=>_ : State -> State -> Set where

=>PUSH : {n k : Nat} {p : Code n} {r : Vec Nat n} {s : Stack} ->
  state n (PUSH k p) r s => state n p r (k :: s)

=>LOAD : {n : Nat}{i : Fin n}{p : Code n}{r : Vec Nat n}{s : Stack} ->
  state n (LOAD i p) r s => state n p r (lookup i r :: s)

=>ADD : {n : Nat}{p : Code n}{r : Vec Nat n}{s : Stack}{x y : Nat} ->
  state n (ADD p) r (x :: y :: s) => state n p r ((y + x) :: s)
```

One rule per instruction; *no rule for HALT*, so a state with code HALT is final. Note ADD pops x then y and pushes y + x: the operand pushed *first* is the deeper one

The compiler

```
compile-aux : {n : Nat} → Exp n → Code n → Code n
compile-aux (lit k)   acc = PUSH k acc
compile-aux (var i)   acc = LOAD i acc
compile-aux (add e f) acc = compile-aux e (compile-aux f (ADD acc))

compile : {n : Nat} → Exp n → Code n
compile e = compile-aux e HALT
```

Written with an *accumulator*: `compile-aux e acc` is the code which evaluates `e`, pushes its value, and *then continues with `acc`*

This is what avoids having to reason about *concatenation* of code — and, as we shall see, it is exactly what makes the induction work

What it produces

```
testcompile :  
  compile exmp ≡ LOAD zero (PUSH one (ADD (LOAD zero (ADD HALT))))  
testcompile = refl
```

For `exmp = add (add (var zero) (lit one)) (var zero)`: load x,
push 1, add, load x, add, halt

Again `refl`: the compiler is a *function*, so Agda runs it

What correctness should say

We would like:

```
state n (compile e) r [] =>* state n HALT r (eval e r :: [])
```

Running the compiled code gives the value the source denotes. It relates the *denotational* semantics of the source with the *operational* semantics of the target

But try to prove it by induction on e . In the case `add e0 e1` the code `compile e0` is *not* run with an empty stack, and it is *not* followed by `HALT`. The induction hypothesis does not apply

The statement the induction can carry

```
correct-aux : {n : Nat} (e : Exp n)(r : Vec Nat n)
              (s : Stack)(acc : Code n)
  → state n (compile-aux e acc) r s
  =>* state n acc r (eval e r :: s)
```

Generalised over *the stack s* and *the continuation acc*: “whatever is already on the stack, and whatever is to be done afterwards, the compiled code of e pushes the value of e and leaves the rest alone”

Finding this statement is the proof. The rest is routine

The two easy cases

```
correct-aux {n} (lit k) r s acc = oneStep =>PUSH  
correct-aux {n} (var i) r s acc = oneStep =>LOAD
```

For `lit k`: the code is `PUSH k acc`, one step gives state `n acc r (k :: s)`, and `eval (lit k) r` *computes* to `k`

The definitional equality does the bookkeeping: we never have to say that `eval (lit k) r = k`

The case of add

```
correct-aux {n} (add e0 e1) r s acc = trans run0 (trans run1 step)
where
  run0 : state n (compile-aux e0 (compile-aux e1 (ADD acc))) r s
        =>* state n (compile-aux e1 (ADD acc)) r (eval e0 r :: s)
  run0 = correct-aux e0 r s (compile-aux e1 (ADD acc))

  run1 : state n (compile-aux e1 (ADD acc)) r (eval e0 r :: s)
        =>* state n (ADD acc) r (eval e1 r :: eval e0 r :: s)
  run1 = correct-aux e1 r (eval e0 r :: s) (ADD acc)

  step : state n (ADD acc) r (eval e1 r :: eval e0 r :: s)
        =>* state n acc r (eval e0 r + eval e1 r :: s)
  step = oneStep =>ADD
```

Two recursive calls, then one ADD. Each run is *named*, with the state it starts from and the state it reaches

Reading the three runs

The goal starts from state n `(compile-aux (add e0 e1) acc) r s`, and `run0` starts from state n `(compile-aux e0 (compile-aux e1 (ADD acc))) r s`: these are *definitionally* equal, Agda unfolds `compile-aux` by itself

`run0`: the induction hypothesis for `e0`, with continuation `compile-aux e1 (ADD acc)` and *the same stack*

`run1`: the induction hypothesis for `e1`, with continuation `ADD acc` and the stack *now one deeper*. This is where the generalisation over `s` pays

`step`: one machine step, and $y + x$ is `eval e0 r + eval e1 r` — the order is right because `e0` was pushed first. The goal ends in `eval (add e0 e1) r :: s`, again by unfolding `eval`

The theorem

```
correct-compile : {n : Nat} (e : Exp n)(r : Vec Nat n)(s : Stack)
  → state n (compile e) r s =>* state n HALT r (eval e r :: s)
correct-compile e r s = correct-aux e r s HALT
```

The theorem we wanted is the *special case* $\text{acc} = \text{HALT}$ of the statement we could prove

This is the usual shape of an inductive proof about a compiler: strengthen until the induction closes, then instantiate

What we relied on

- the source semantics is a *function* (`eval`), the target semantics is a *relation* (`=>`). Correctness relates the two
- the compiler is a function too, so Agda *computes* with it: we never say that `compile-aux (add e0 e1) acc` unfolds
- the *accumulator* style: no code concatenation, hence no lemma about it
- the generalisation over `s` and `acc`: the real content

For a real compiler, replace `Exp` by `C` and `Code` by assembly; the shape of the proof does not change, only its size

Where does the accumulator come from?

The way one writes a compiler first is with *concatenation*

```
compile (add e f) = compile e ++ compile f ++ (ADD :: [])
```

and then the proof needs a lemma: running $p \mathrel{++} q$ is running p , then running q . This is how the early proofs go

The accumulator is *later*: it is Wand's idea of compiling into a *continuation* (1982), made systematic by Hutton and Bahr, *Calculating correct compilers* (2015)

It is the same move as generalising over s : say what the code of e does *in an arbitrary context*, and the lemma about $++$ disappears

For whom are the names?

```
correct-aux {n} (add e0 e1) r s acc =  
  trans (correct-aux e0 r s (compile-aux e1 (ADD acc)))  
    (trans (correct-aux e1 r (eval e0 r :: s) (ADD acc))  
      (oneStep =>ADD))
```

The same theorem, the *same term*, without the where (compiler2.agda). The three type signatures were doing nothing for Agda: it infers every intermediate state

Ask a language model for the proof and you get *this* version — it has no need to see the intermediate states, and none for implicit arguments either. The names and the stated states are *for a reader*, not expressive power

What survives is the only thing that was ever difficult: *generalising over s and acc*

What to remember

- three ways to give a semantics: *small step*, *big step*, *denotational*; each is a definition in Agda, and the theorems relate them
- a *relation* is an indexed inductive family; a proof about it is a function defined by pattern matching on that family
- safety is *progress + preservation*, and says exactly that the machine is never at a loss
- in a compiler proof, the work is in *strengthening the statement* until the induction goes through

Exercises, second hour

1. Prove the converse of sound $\Downarrow$: $e \Rightarrow^* v$ with $\text{isVal } v$ gives $e \Downarrow v$. First: $e \Rightarrow e'$ and $e' \Downarrow v$ give $e \Downarrow v$
2. Write down exactly where the induction on e breaks for the un-generalised statement of `correct-aux`
3. Add `mul` to `Exp` and `MUL` to `Code`, and extend the proof. How many lines change?
4. Define the machine as a *function* `run : Nat -> State -> State` and redo the proof. Which version is easier?
5. Compile the language of the *first* hour, with `if`

References

- A. Turing. *Checking a large routine*, 1949
- J. McCarthy, J. Painter. *Correctness of a compiler. . .*, 1967
- R. Milner, R. Weyhrauch. *Proving compiler correctness. . .*, 1972
- R. Milner. *A theory of type polymorphism in programming*, 1978
- G. Plotkin. *A structural approach to operational semantics*, 1981
- B. Pierce. *Types and Programming Languages*, 2002, chapter 8
- X. Leroy. *Formal certification of a compiler back-end*, 2006

Files (.agda): arith1, arithexp, operational, compiler, compiler2